Kiến trúc Đệ quy Bottom-up và Cơ chế Bảo toàn File Descriptor — (du.c)

1. Quét Entry Thư mục

Truy xuất Inode & Lọc nhiễu

Kiểm tra stat(path, &st):

- T_FILE / DEVICE: in & trả về st.size
- T_DIR: Mở fd = open(path, 0)

Bộ đệm mảng động (names):

```
char (*names)[15] = malloc(...);
```

- Bỏ qua de.inum == 0 (slot trống)
- Bỏ qua "." và ".." (ngừa lặp vô hạn)
- Nhân đôi mảng khi count >= capacity

✓ Thu thập toàn bộ tên con vào RAM

2. Điểm sáng: Đóng FD Sớm

Phòng tránh triệt để lỗi cạn kiệt 16 FDs của xv6

LỆNH THEN CHỐT:

```
close(fd); /* NGAY LẬP TỨC */
```

Đóng FD trước khi bước vào đệ quy!

Giới hạn khắt khe của hệ điều hành:

- xv6 giới hạn cứng mỗi tiến trình: NOFILE = 16.

✗ **Nếu giữ fd mở trong đệ quy:**
Cây thư mục lồng nhau sâu > 13 tầng sẽ gây lỗi cạn kiệt FD ("cannot open"), sập đệ quy.

✓ **Khi đóng fd sớm (giải pháp của bài làm):**
Số FD mở đồng thời tại mỗi thời điểm chỉ là O(1) (tối đa 1 FD cho thư mục đang quét danh sách).
✓ An toàn tuyệt đối với mọi độ sâu cây thư mục.

Vòng đời FD được tối ưu hóa:

```
[open] → [đọc hết vào names] → [close]
```

3. Đệ quy Bottom-up & Tổng hợp

Cộng dồn kích thước cây & Điều kiện in theo cờ

Vòng lặp đệ quy (Bottom-up Post-order):

```
for (int i = 0; i < count; i++) {
   total_size += du(path + "/" + names[i], a, s, 0);
}
```

free(names); // Giải phóng heap mảng con khi tính xong

Chế độ xuất kết quả theo các cờ tùy chọn:

1. Mặc định: du . (Chỉ in thư mục)
In thư mục con khi hoàn tất đệ quy & thư mục gốc: (!s)

```
64\t./a/aa 96\t./a 0\t./z 96\t.
```

2. Tóm tắt: du . -s (Summary)
Triệt tiêu in thư mục con, chỉ in dòng tổng kết gốc: is_root

```
96\t.
```

3. Chi tiết: du . -a (All entries)
In từng file con (st.type == T_FILE && a && !s) và thư mục

```
16\t./a/f1.txt 32\t./a/aa/f2.txt 96\t.
```